⚡ C++ Quick Revision & Pre-Interview Cheat Sheet

Revision Goal: 45-second per-topic recall covering **Problem**, **Internals**, **Complexity**, **Trade-offs**, and **Interview Essentials**.

🔬 Module 1: Execution Mechanics & I/O Streams

1. `\n` vs `std::endl`

- **Problem**: Formatting output stream buffers.
- **Internals**: `\n` appends a character byte to RAM buffer ($O(1)$ user-space). `std::endl` appends `\n` AND executes `std::cout.flush()`, triggering a POSIX `sys_write` kernel context switch.
- **Complexity**: `\n` is $O(1)$ fast; `std::endl` in heavy loops causes extreme system slowdown.
- **Trade-off**: Use `\n` for performance; use `std::endl` only for crash-critical real-time logging.
- **Interview Q**: *Why avoid* `std::endl` *in loops?* Forces an OS system call flush on every iteration.

2. Primitive Data Types & Implicit Promotion

- **Problem**: Storing scalar numerical, character, and truth values.
- **Internals**: `char` (1B), `bool` (1B), `int` (4B, 2's complement), `double` (8B, IEEE 754).
- **Promotion Rule**: Mixed operations promote lower-rank types to higher-rank types (`int` + `double` $\rightarrow$ `double`). Integer division (`5 / 2`) truncates to `2`.
- **Interview Q**: *Result of* `5 / 2` *vs* `5 / 2.0`*?* `2` (`int`) vs `2.5` (`double`).

3. Guard Clauses & Short-Circuiting (`&&`, `||`)

- **Problem**: Preventing unnecessary checks and unallocated memory dereferences.
- **Internals**: `false && expr` and `true || expr` terminate evaluation instantly without reading `expr`.
- **Trade-off**: `if (ptr != nullptr && ptr->isValid())` prevents runtime Null Pointer Dereferences.

4. Selection Mechanics: `switch` vs `if-else`

- **Problem**: Branching execution on discrete integer/enum values.
- **Internals**: Sequential `if-else` checks conditions $O(N)$. `switch` compiles into a **Jump Table** array for direct $O(1)$ instruction jumps.
- **Trade-off**: Omitting `break` causes intentional/accidental fallthrough. Always include `default:`.

🧮 Module 2: Arrays & Spatial Matrices

1. 1D Stack Arrays & Array Decay

- **Problem**: Contiguous stack storage for fixed-size elements.
- **Internals**: Array identifier (`arr`) implicitly decays to a raw pointer (`&arr[0]`) when passed to functions, stripping size information.
- **Trade-off**: Out-of-bounds access (`arr[N]`) corrupts stack memory (Segmentation Fault).

2. 2D Matrices & Row-Major Stride Mapping

- **Problem**: Representing multi-dimensional grids in 1D contiguous RAM.
- **Internals**: Row-Major Offset Formula: $\text{Offset} = (r \times \text{Cols}) + c$.
- **Key Properties**:
  - Main Diagonal: $r == c$ (`matrix[i][i]`).
  - Skew-Symmetric ($A^T = -A$): Diagonal elements **must be** 0, and $A[i][j] == -A[j][i]$.
- **Param Rule**: Parameter arrays must specify all inner dimensions (e.g., `void f(int arr[][3][3])`) so the compiler can calculate stride byte offsets.

🔤 Module 3: Strings, STL Algorithms & References

1. Two-Pointer String Mechanics

- **Problem**: Inspecting or mutating string buffers in-place.
- **Internals**: Two converging index pointers (`left` , `right`) check palindromes in $O(N)$ time and $O(1)$ space.
- **Trade-off**: `for (char& c : str)` mutates memory in-place without temporary object allocation.

2. Parameter Passing Semantics

- **Pass-by-Value (`T`)**: Deep copy ($O(N)$ memory/time cost).
- **Pass-by-Ref (`T&`)**: Alias to original memory ($O(1)$ cost, allows caller mutation).
- **Pass-by-Const-Ref (`const T&`)**: Read-only alias ($O(1)$ cost, prohibits mutation — Gold Standard for containers).

3. `std::vector`

- **Problem**: Dynamic contiguous array on the heap.
- **Internals**: Managed by 3 stack pointers (`_first` , `_last` , `_end`). Doubling capacity allocation ($1.5\times$ / $2\times$) when `size == capacity` .
- **Complexity**: Access $O(1)$, Append Amortized $O(1)$, Insert/Delete Middle $O(N)$.
- **Pros**: Maximum CPU L1/L2 cache line hits due to contiguous memory layout.
- **Cons**: Reallocation invalidates all existing pointers, references, and iterators.

- **Optimizations**:
  - `.reserve(N)` : Pre-allocates raw capacity without constructing objects (prevents reallocations).
  - `.emplace_back()` : Constructs elements in-place inside vector memory without temporary copy/move operations.

🗺️ Module 4: Associative Containers & Classes

1. `std::map` vs `std::unordered_map`

- `std::map` : Red-Black Tree. Strict $O(\log N)$ lookups. Keys remain sorted.
- `std::unordered_map` : Hash Table with bucket array chaining. Average $O(1)$ lookups.
- **Worst-case $O(N)$ Risk**: Severe hash collisions turn bucket lookups into linear list traversals.
- `operator[]` **Danger**: Default-constructs missing keys automatically! Use `.find()` , `.count()` , or `.at()` for safe read lookups.
- `.try_emplace()` : Peak insertion method that constructs values in-place inside the hash bucket only if the key is missing.

2. OOP & Object Initialization

- `struct` vs `class` : `struct` defaults to `public:` access; `class` defaults to `private:` .
- **Member Initializer List**: Constructs fields directly before constructor execution (`Class(string n) : name(n) {}`).
- **Default Constructor Trap**: Declaring *any* custom constructor suppresses compiler generation of `Class()` . Fix: `Class() = default;` .

⚡ Module 5: Dynamic Memory & Move Semantics

1. Stack vs Heap & Raw Memory

- **Stack**: Fast, CPU-managed LIFO scope frame allocation.
- **Heap**: Dynamic memory pool managed via `new` / `delete` . Outlives call stack scopes.
- **Memory Leak**: Dynamic heap allocation orphaned without `delete` .
- **Array Deletion Rule**: `new T[N]` **must** be freed using `delete[] ptr` . Scalar `delete ptr` leaks destructors for elements $1 \dots N-1$.
- **Dangling Pointer**: Dereferencing freed memory. Always assign `ptr = nullptr` post-deletion!

2. Move Semantics (`std::move`)

- **Problem**: Eliminates deep-copy penalties for temporary rvalue objects.
- **Internals**: `std::move(x)` casts `x` to an rvalue reference (`T&&`), allowing destination objects to "steal" heap pointers directly.

🏛️ Module 6: Inheritance & Dynamic Polymorphism

1. Virtual Functions & VTables

- **Problem**: Decouples interfaces from implementations, allowing runtime method dispatch via base pointers (`Base*`).
- **Internals**: Class objects with `virtual` methods hold an 8-byte pointer (`vptr`) pointing to a static class function lookup table (**VTable**).
- **Trade-off**: Virtual calls add indirect pointer dereference overhead and cannot be inlined by compilers.
- **Mandatory Rule**: Base class destructors **must** be `virtual` (`virtual ~Base()`) to prevent resource leaks and Undefined Behavior when deleting derived objects via base pointers.
- **Object Slicing**: Passing derived objects by value to base parameters strips derived fields and VTables.

🛡️ Module 7: Smart Pointer Suite (RAII)

1. `std::unique_ptr`

- **Ownership**: Exclusive single owner. Copying disabled (`= delete`); transfer via `std::move()` .
- **Overhead**: Zero runtime abstraction cost (0 extra bytes over raw pointer).

2. `std::shared_ptr`

- **Ownership**: Shared reference-counted ownership.
- **Internals**: Managed via a dynamic **Control Block** (strong count, weak count, custom deleter).
- **Optimization**: `std::make_shared<T>()` allocates object and Control Block in a single contiguous memory block.

3. `std::weak_ptr` (Circular Dependency Fix)

- **Problem**: Solves mutual pointer loops (e.g., Parent ↔ Child or Order ↔ Receipt) that freeze reference counts above zero permanently.
- **Internals**: Points to Control Block without incrementing strong reference count.
- **Access Protocol**: Invoke `.lock()` to temporarily elevate `weak_ptr` to a valid `shared_ptr` before reading data.

🎯 15-Second Interview Decision Matrix

Requirement	Best Tool	Key First-Principles Reason
Dynamic array, max iteration speed	`std::vector<T>`	Contiguous memory = CPU L1/L2 cache line pre-fetching

Fast key-value search	`std::unordered_map`	$O(1)$ average hash table access
Key-value store, sorted output	`std::map`	$O(\log N)$ Red-Black Tree order
Single owner, memory safety	`std::unique_ptr<T>`	Zero performance overhead over raw pointers
Shared resource lifecycle	`std::shared_ptr<T>`	Reference-counted Control Block management
Parent back-reference / cycle fix	`std::weak_ptr<T>`	Non-owning observer; inspect via `.lock()`